

SK네트웍스 Family AI 과정 23기

**모델링 및 평가 LLM 활용 소프트웨어**

|        |                                                                                                                                           |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------|
| 산출물 단계 | 모델링 및 평가                                                                                                                                  |
| 평가 산출물 | LLM 활용 소프트웨어                                                                                                                              |
| 제출 일자  | 2026-05-01                                                                                                                                |
| 깃허브 경로 | <a href="https://github.com/SKNETWORKS-FAMILY-AICAMP/SKN23-FINAL-3Team">https://github.com/SKNETWORKS-FAMILY-AICAMP/SKN23-FINAL-3Team</a> |
| 작성 팀원  | 정유선, 이승연, 송민채                                                                                                                             |

## 1. 시스템 개요

**withDOG** 는 반려견과 함께할 수 있는 장소를 추천해 주는 챗봇(메인 기능)과 AI 그림일기(보조 기능)를 제공하는 LLM 기반 웹 애플리케이션이다. 사용자는 자연어로 챗봇과 대화하고, 시스템은 입력 문장을 의도분류 모델로 분석하여 다이어리 작성·장소 추천·시설정보·잡담의 4가지 의도 중 하나로 분기한 뒤 각 의도에 특화된 LLM 호출과 RAG 검색을 조합해 응답을 생성한다.

본 산출물은 시스템에서 LLM 이 어떤 위치에서 어떻게 활용되는지, 자체 학습한 모델과 외부 호출형 모델을 어떻게 분리·결합했는지, 평가 메트릭과 **baseline** 결과는 어떠한지를 정리한다. 첨부된 LLM 모델 파일은 자체 **fine-tuning** 한 의도분류 모델(KoELECTRA 기반)이며, **OpenAI gpt-4.1-mini · gpt-image-1** 등 외부 호출형 모델은 코드 내 호출 위치와 호출 파라미터를 본문에 명시한다.

프로젝트 진행 과정에서 **LangChain/LangGraph** 도입을 한 차례 보류하고 **OpenAI SDK** 직호출 + 자체 **RAG** 체인으로 단순화하는 결정을 거쳤다. 이는 인력 변동(팀 규모 5인 → 3인) 상황에서 안정성 확보를 위해 의도적으로 선택한 단계이며, 발표 후 **LangChain** 도입과 자체 추론(**Gemma**) 마이그레이션을 고도화 1순위로 계획하고 있다.

## 2. 활용한 LLM 모델

withDOG 시스템에서 사용하는 LLM 및 관련 모델은 아래와 같다. 외부 호출형(OpenAI API)과 자체 학습/사전학습 모델을 의도와 비용 특성에 따라 분리해 사용한다.

| 모델                                             | 유형                 | 용도                                         | 비고                                     |
|------------------------------------------------|--------------------|--------------------------------------------|----------------------------------------|
| OpenAI gpt-4.1-mini                            | 외부 호출형 LLM         | 다이어리 본문 / 장소 추천 답변 / 시설정보 / 잡담 / 자연어 쿼리 파싱 | 운영 통일<br>(2026-04-26 결정)               |
| OpenAI gpt-image-1                             | 외부 호출형 이미지         | 다이어리 그림 (1024×1024, medium)                | 메인 (무료+유료)<br>flat-rate \$0.042/회      |
| OpenAI gpt-image-2                             | 외부 호출형 이미지         | 유료 플랜 그림 서브 옵션 (예정)                        | 토큰 기반 (input \$8 / output \$30 per 1M) |
| KoELECTRA<br>(koelectra-base-v3-discriminator) | 자체 fine-tuning 분류기 | 의도분류 (4 라벨)                                | ★ 첨부 모델 파일 / 1117 샘플 / 정확도 0.9955      |
| jhgan/ko-sroberta-multitask                    | 사전학습 임베딩           | ChromaDB 검색용 384차원 임베딩                     | 사전학습 가중치 그대로 사용 (학습 X)                 |

★ 첨부 LLM 모델 파일: 자체 fine-tuning 한 의도분류 모델(KoELECTRA 기반)을 함께 제출한다. 외부 호출형 모델(OpenAI gpt-4.1-mini · gpt-image-1 · gpt-image-2)은 별도 파일이 존재하지 않으므로 본문에 호출 위치·파라미터를 명시하는 것으로 갈음한다. 한국어 임베딩 모델 jhgan/ko-sroberta-multitask 는 사전학습 가중치를 그대로 사용하며 별도 학습은 진행하지 않았다.

### 2.1 모델 선정 근거

의도분류 모델로 KoELECTRA(monologg/koelectra-base-v3-discriminator)를 채택한 이유는 세 가지이다. 첫째, 매 사용자 메시지마다 외부 LLM 을 호출하면 비용·레이턴시가 누적되며, 4 라벨 단순 분류 작업은 LLM 호출이 과대 기능에 해당한다. 둘째, ELECTRA 계열은 BERT 계열 대비 동일 파라미터 수에서 학습 효율이 높고, 한국어 코퍼스 기반 사전학습 모델인

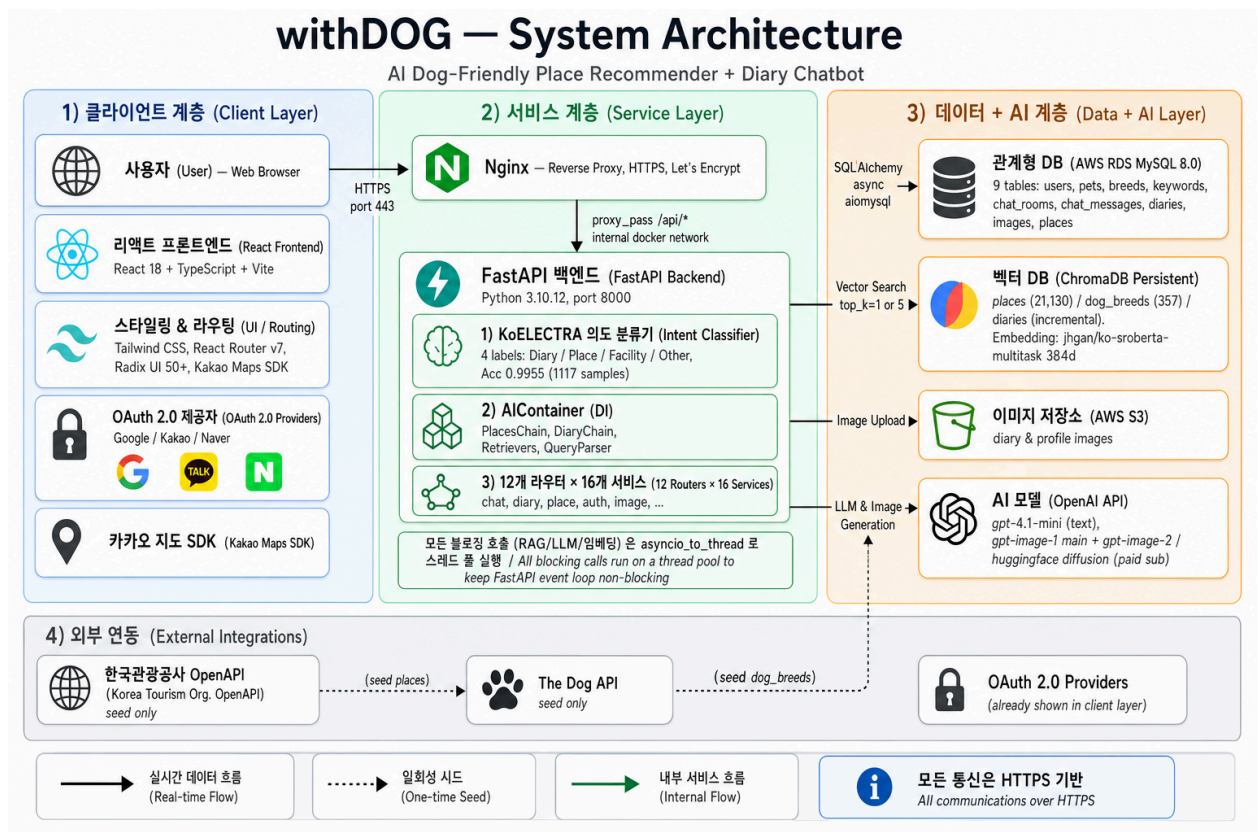
KoELECTRA 는 분류 태스크에서 강한 베이스라인으로 알려져 있다. 셋째, 향후 온프레미스 또는 자체 추론 환경으로 옮길 가능성을 고려해 의도 분류만 별도 모델로 분리했다.

본문 생성·답변 생성에는 OpenAI gpt-4.1-mini 를 사용한다. 다이어리 본문(temperature 0.4 ~ 0.85)·장소 추천 답변·시설정보 응답·잡담 응답·자연어 쿼리 파싱까지 모든 텍스트 생성 호출이 동일 모델로 통일되어 있어 비용 추적과 운영이 단순하다. 이미지 생성은

gpt-image-1 (1024×1024, medium quality, quality flat-rate \$0.042/회) 을 메인으로 사용하며, 향후 유료 플랜 사용자에게는 gpt-image-2(토큰 기반) 와 huggingface 호스팅 diffusion 계열 모델(FLUX/Stable Diffusion)을 서브 옵션으로 제공할 계획이다.

### 3. 시스템 아키텍처

withDOG 의 LLM 활용 흐름은 단일 진입점(POST /api/chat-rooms/{id}/messages)에서 시작해 의도분류 → 의도별 핸들러 → RAG 검색 → LLM 호출 → DB 영속화의 순서로 처리된다. 모든 LLM 및 ChromaDB 호출은 비동기(asyncio)로 처리되며, 블로킹 호출은 `asyncio.to_thread()` 또는 `run_in_executor()` 로 스레드 풀에 위임해 이벤트 루프 차단을 방지한다.



[그림 1] withDOG 시스템 — 클라이언트부터 OpenAI / ChromaDB / RDS / S3 까지 전체 데이터 흐름

다이어그램은 챗봇 메시지 한 번의 처리 흐름을 보여준다. 클라이언트(React)가 Nginx 를 거쳐 FastAPI 백엔드에 요청을 보내면, `intent_service` 가 KoELECTRA 로 4 라벨 중 하나를 분류한 뒤, 해당 의도의 핸들러(`_handle_diary` / `_handle_places` / `_handle_facility` / `_handle_fallback`)가 호출된다. 다이어리·장소·시설정보 의도는 ChromaDB 임베딩 검색을 거쳐 컨텍스트를 보강한 뒤 OpenAI gpt-4.1-mini 로 답변을 생성하며, 잡담 의도는 RAG 없이 LLM 직답으로 응답한다. 이미지 생성이 필요한 다이어리 작성 흐름은 텍스트 생성과 이미지 생성을 두 단계로 분리(2-phase)해 사용자가 본문을 먼저 보고 이미지는 비동기로

받아볼 수 있도록 설계했다.

DI 컨테이너(ai/container.py:AIContainer)는 SentenceEmbedder · ChromaVectorStore · OpenAICostTracker · AsyncOpenAI · BreedRetriever · DiaryRetriever · PlacesRetriever · DogScorer · OwnerScorer · DiaryPromptBuilder · PlacesPromptBuilder · DiaryChain 을 앱 시작 시 1회 인스턴스화해 app.state.ai\_container 에 보관한다. 이를 통해 매 요청마다 임베더·벡터 스토어를 재생성하는 비용을 제거하고, 체인 간 공통 인프라(임베더, ChromaDB, 비용 추적)를 공유하면서도 체인 자체는 독립적으로 동작하도록 분리했다.

백엔드 lifespan 부팅 순서는 ① 로깅 초기화(RotatingFileHandler 10MB×5) ② SSH 터널 (로컬 환경에서 EC2 경유 RDS 포트 개방) ③ DB 엔진 (aiomysql, pool\_size 10, recycle 1800s, pre\_ping) ④ APScheduler (매일 03:00 KST 탈퇴 사용자 hard delete) ⑤ KoELECTRA 워밍업(warmup\_intent\_model, 실패 시 lazy-load 폴백) ⑥ 커스텀 건중 upsert ⑦ AIContainer 조립의 7단계로 구성된다.

## 4. LLM 활용 방식

### 4.1 의도분류 — KoELECTRA

사용자가 챗봇에 메시지를 보내면 가장 먼저 KoELECTRA 의도분류기가 입력 문장을 4 라벨(다이어리 작성·장소추천·시설정보·기타) 중 하나로 분류해 의도별 핸들러로 분기한다. 모델은 monologg/koelectra-base-v3-discriminator 를 베이스로 ElectraForSequenceClassification 분류 헤드를 신규 초기화해 fine-tuning 했다.

학습 데이터는 ai/intent/data/sample\_data.csv 에 1202 샘플(시설정보 382 / 다이어리 작성 277 / 장소추천 281 / 기타 262, max/min 비율 1.46× 학습 안전 범위) 이 있으며, 학습/검증 분할 8:2 (893/224, 1117 샘플 시점 기준), 하이퍼파라미터는 batch\_size=16 · epochs=5 · learning\_rate=3e-5 · max\_length=128 tokens 이다. 2026-04-28 GPU 재학습 결과 학습셋 정확도 0.9955 를 확보했다.

모델 파일은 ai/intent/model/ 디렉토리에 model.safetensors · config.json · tokenizer 관련 파일로 저장되며, 운영 시에는 백엔드 lifespan 단계에서 warmup\_intent\_model() 로 사전 로드해 첫 요청 지연을 제거한다. intent\_service.classify\_intent\_async() 가 입력 문장을 토큰나이징해 모델에 전달하고 logits argmax 로 라벨을 결정한다.

### 4.2 다이어리 생성 — DiaryChain (gpt-4.1-mini + gpt-image-1)

다이어리 작성 의도로 분류된 메시지는 DiaryChain.run() 으로 라우팅된다. DiaryChain 의 처리 단계는 다음과 같다.

BreedRetriever.get\_breed\_context(breed) — 견종 메타정보 1건을

ChromaDB(dog\_breeds 컬렉션, 357종)에서 검색.

DiaryRetriever.search(conversation\_summary, user\_id, n\_results=3) — 사용자 본인의 유사 과거 일기 3건을 검색. user\_id 필터로 타인의 일기 노출을 차단(프라이버시).

DiaryPromptBuilder.build\_diary\_prompt(...) — 12개 감정 이모지 → EMOTION\_TONE 매핑(😊 밝고 신나는 / 😊 잔잔한 / 😊 몽클한 / 😊 나른한 / ❤️ 다정한 등), 4개 일기 유형 → DIARY\_TYPE\_FOCUS(강아지 시점 / 보호자 시점 / 특별한 순간 / 일상), 야간/해외 키워드 자동 감지 분기를 거쳐 시스템·사용자 프롬프트 조립.

AsyncOpenAI.chat.completions.create(model='gpt-4.1-mini', temperature=0.4~0.85, max\_tokens=1500) — JSON 출력 강제 ({title, content, summary, image\_prompt\_base}).

DiaryPromptBuilder.build\_final\_image\_prompt(image\_prompt\_base, breed\_en, ...) — 영문 90단어 이내, 견종별 시각 디테일(BREED\_EN\_MAP 40+종, BREED\_VISUAL\_HINTS 20+종) 자동 추가. 예: Poodle → 'clearly curly fluffy coat — NOT straight fur', Dachshund → 'distinctively very long body, very short legs'.

AsyncOpenAI.images.generate(model='gpt-image-1', size=1024, quality='medium') — 그림 생성.

evaluator.complete\_image\_eval() — Subject Purity(40%) · Background Quality(30%) · Style Consistency(30%) 메트릭 자동 계산 후 L0 ~ L5 레벨 분류.

S3 업로드 → image\_id 발급 → diaries 테이블에 PATCH 로 image\_id 바인딩.

텍스트 생성과 이미지 생성을 두 단계로 분리한 **2-phase** 구조 덕분에 사용자는 본문을 먼저 화면에 받아보고 이미지는 비동기로 표시되며, 다이어리 본문은 일기 데이터로 즉시 영속화되고 이미지는 추후 image\_id 가 채워지는 형태로 부분 실패에 대한 회복성을 확보했다.

### 4.3 장소 추천 — RAG 검색 + gpt-4.1-mini

장소추천 의도로 분류된 메시지는 place\_service 와

chat\_response\_service.generate\_place\_reasons 로 라우팅된다. 처리 단계는 다음과 같다.

QueryParser.\_parse\_query\_with\_llm — 자연어 쿼리를 11개 구조화 키(city · district · sub\_category · is\_indoor · is\_outdoor · has\_parking · pet\_zone\_type · pet\_size\_limit · entrance\_fee\_preference · extra\_fee\_preference · time\_condition) + landmark + subjective 로 변환. 동네명 → 구 매핑(잠실→송파구, 홍대→마포구, 압구정/청담→강남구) 사전 + Kakao Local API landmark 좌표화(1000m → 2000m → 3000m 단계 확장 검색)를 사용.

DogScorer.classify\_type(dog\_tags) / OwnerScorer.classify\_type(owner\_tags) — 24개 강아지 태그 / 19개 보호자 태그를 5차원 점수 벡터(a 활동성 / b 사회성 / c 순종 / d 호기심 / e 자극민감도)로 환산해 d\_a~d\_d, o\_a~o\_d 타입 분류.



`PlacesRetriever.search(query, category, city, n_results=5)` — ChromaDB(dog\_places 컬렉션, 21,130 건)에서 메타 필터(category, city) 적용 후 임베딩 유사도 검색. 메타 필드 17종 보존(name, address, lat, lng, open\_hours, parking, entrance\_fee, indoor, outdoor, size\_limit, conditions 등).

재순위(rerank) — 반려견 5차원 성향 벡터로 score 가산/감산. 예: e>3 (자극민감) + 실외 장소 → -0.1 / a>3 (활동성) + 공원 → +0.1 / b>3 (사회성) + 카페 → +0.05.

`generate_place_reasons` (chat\_response\_service.py:289-331) — 후보 장소 리스트와 사용자 쿼리·반려견 정보를 입력으로 gpt-4.1-mini 가 장소별 추천 이유 1문장 생성. \_PLACE\_REASON\_SYSTEM\_PROMPT 6행 가드 + inline 8행 가드로 hallucination 방지(데이터에 없는 운영 정책·시설 정보 추정 금지).

임베딩 모델 jhgan/ko-sroberta-multitask 는 한국어 코퍼스 기반 RoBERTa 384차원 임베딩으로, ChromaDB(HNSW cosine similarity) 위에서 동작한다. 단순 임베딩 검색의 한계(공간·시간·긍정부정·일반문구 중복)는 RDB 메타 필터 + landmark radius 하이브리드로 보완하며, 입장료 정규화는 `places.entrance_fee_ * 4` 컬럼(20,748건 RDB UPDATE 완료) + Chroma metadata 동기화로 ETL 단계에서 처리한다.

#### 4.4 시설정보 응답 — 단일 시설 RAG + 결정적 답변

시설정보 의도(예: '홍대 000 카페 운영시간 알려줘')로 분류된 메시지는 단일 시설 정확 매칭(name LIKE) 후 ChromaDB fallback, 그리고 속성 슬롯 분류(7종: 운영시간 / 주차 / 입장료 / 반려견 동반 조건 / 위치 / 편의시설 / 일반)로 처리된다. 답변은 속성별 결정적 응답 형태로 LLM 미사용이며, 카드 페이로드(ChatTurnResponse.facility) 로 프론트엔드에 전달되어 챗봇 텍스트 버블이 아닌 별도 카드 UI 로 표시된다.

이 분기는 LLM 의 hallucination 리스크를 원천적으로 제거하기 위해 의도적으로 LLM 호출을 배제했다. 시설 운영시간·주차 정보 등 정확성이 중요한 사실 질의에 대해 LLM 이 자유 생성하면 데이터에 없는 정보를 만들어낼 위험이 있어, RDB 메타데이터를 그대로 표기하는 결정적 답변 흐름으로 설계했다.

#### 4.5 잡담 응답 — gpt-4.1-mini 직답

기타(잡담) 의도로 분류된 메시지는 RAG 검색을 거치지 않고 gpt-4.1-mini 가 직접

응답한다. 인사·일반 잡담·도메인 외 질문이 여기에 해당하며, 시스템 프롬프트에 withDOG  
의 페르소나(반려견 동반 여행 전문 챗봇)와 응답 규칙(개인정보 수집 금지·욕설  
차단·서비스 범위 외 질문은 부드럽게 제한)이 박혀 있다.

## 5. RAG 구조

withDOG 의 RAG(Retrieval-Augmented Generation)는 ChromaDB Persistent Client(sqlite3 기반, HNSW cosine similarity) 위에 3개 컬렉션을 운영한다.

| 컬렉션         | 건수           | 임베딩 대상 텍스트                           | 메타 필드                                                                                                                              |
|-------------|--------------|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| dog_breeds  | 357          | 견종 이름 + 한글명 + 활동성 등급 + 기질 텍스트        | breed_name,<br>breed_name_ko,<br>activity_level,<br>temperament                                                                    |
| dog_places  | 21,130       | 시설 특징 + 분위기 + 운영정보 등<br>8개 섹션 자연어 문장 | name, address,<br>category, lat, lng,<br>open_hours, parking,<br>entrance_fee, indoor,<br>outdoor, size_limit,<br>conditions (17종) |
| dog_diaries | 증분<br>(사용자별) | 다이어리 본문 + 요약                         | id, user_id, diary_date,<br>location, summary                                                                                      |

임베딩 모델은 `jhgan/ko-sroberta-multitask`(384차원)로 통일되어 있어 컬렉션 간 임베딩 차원이 일치한다. 컬렉션은 호스트 볼륨(`data/chroma_db/`)에 영속화되어 컨테이너 재시작 시에도 인덱스가 유지된다.

RAG 검색의 정확도를 높이기 위해 임베딩 단독 검색 대신 RDB 메타 필터(`category · city · entrance_fee_type` 등)와 `landmark` 좌표 기반 거리 필터를 결합한 하이브리드 구조를 사용한다. 멘토 04-21 피드백에 따라 임베딩이 갖는 한계 4종(공간 개념 부재 · 시간 개념 부재 · 긍정/부정 구분 미약 · 일반 문구 중복)을 메타 필터로 보완한 결과이며, 평가 메트릭(아래 §6)으로 효과를 정량 측정한다.

## 6. 평가 결과

장소 검색 RAG 의 품질을 정량 측정하기 위해 자체 평가 모듈(ai/evaluation/, 7개 파일 451 라인)을 구축했다. 평가셋 150건(retrieval 118 + refusal 32) 을 9개 카테고리로 분류해 Hit@5 · Recall@5 · Refusal Rate 메트릭을 측정한다.

### 6.1 평가셋 카테고리 (150건)

| 카테고리      | 건수  | 평가 방식                      |
|-----------|-----|----------------------------|
| 복합 조건     | 29  | Hit@5, Recall@5            |
| 지역 × 장소유형 | 27  | Hit@5, Recall@5            |
| 주관 형용사    | 21  | Hit@5, Recall@5            |
| 위치 기반 근처  | 16  | Hit@5, Recall@5            |
| 동반 조건 특수  | 11  | Hit@5, Recall@5            |
| 실내/실외     | 11  | Hit@5, Recall@5            |
| 시간 조건     | 11  | Hit@5, Recall@5            |
| 편의시설 (주차) | 9   | Hit@5, Recall@5            |
| 오류/범위 밖   | 32  | Refusal Rate (서울 외 거절)     |
| 합계        | 150 | retrieval 118 + refusal 32 |

### 6.2 메트릭 정의

Hit@5 — Top-5 안에 정답이 하나라도 있으면 1, 없으면 0 (이진 평가).

Recall@5 — 정답이 여러 개일 때 Top-5 에 포함된 비율 (0.0 ~ 1.0).

Refusal Rate — 서울 외 지역 등 범위 밖 질문을 올바르게 거절한 비율.

### 6.3 Baseline 결과 (RUN\_20260426)

| 메트릭                  | 값              | 비고    |
|----------------------|----------------|-------|
| Hit@5 (전체 retrieval) | 0.5254 (52.5%) | n=118 |

| 메트릭                       | 값              | 비고                            |
|---------------------------|----------------|-------------------------------|
| Recall@5 (전체 retrieval)   | 0.4102 (41.0%) | n=118                         |
| Refusal Rate (전체 refusal) | 0.5938 (59.4%) | n=32 / 서울 외 거절 정확도            |
| [취약] 위치 기반 근처 Hit@5       | 0.250 (25.0%)  | n=16 / landmark 추출·구 매핑 개선 필요 |
| 복합 조건 Hit@5               | 0.483 (48.3%)  | n=29                          |

위치 기반 근처 카테고리(Hit@5 25.0%) 가 가장 약한 것으로 측정되었으며, 원인은 query\_parser 의 landmark 추출 정확도와 동네명 → 구 매핑 사전의 커버리지가 핵심 개선 포인트로 식별되었다. Refusal Rate(59.4%) 는 서울 외 city 검출 정확도 향상(목표 80%+) 과 Fallback 응답의 hallucination 가드 패치(\_PLACES\_SYSTEM\_PROMPT 에 '데이터에 없는 운영 정책 추정 금지' 1줄 추가)로 회복 가능하다.

평가 실행 명령은 ``python ai/evaluation/run_evaluation.py [--limit N] [--category 카테고리명] [--output 파일경로]`` 이며, 결과는 `evaluation/results/eval_results_YYYYMMDD_HHMMSS.xlsx` 와 `evaluation/logs/run_YYYYMMDD_HHMMSS.log` 로 저장된다. run\_id 컬럼으로 여러 실행을 구분할 수 있어 개선 전후 비교가 용이하다. 본 산출물에는 RUN\_20260426 baseline 결과 xlsx 를 부속 자료로 동봉한다.

## 7. 향후 계획

발표 후 1순위로 진행할 LLM 활용 고도화 항목은 다음과 같다.

LangChain LCEL · LangGraph 도입 — 현재 OpenAI SDK 직호출 + 자체 체인 구조를 LCEL pipe 표기와 LangGraph 워크플로로 마이그레이션. 장기적으로 Tool Calling 및 멀티 에이전트 분기를 가능하게 함.

Gemma 자체 추론 마이그레이션 — 멘토 04-21 의 '로컬 모델 도입' 권고 정렬.

비용·온프레미스 가능성 확보. 평가셋 150건을 동일 기준으로 비교해 마이그레이션 범위 결정.

평가 지표 다양화 — Top N (1·3·5·10·20) 다양화 + RAG only / RDB only / 결합 비교 실험으로 메트릭 설득력 강화. 강사 04-30 피드백 정렬.

이미지 모델 다층 전략 완성 — 메인 gpt-image-1 (무료+유료) 유지하면서 유료 플랜에 gpt-image-2 + huggingface diffusion (FLUX/SD) 서브 옵션 분기. ComfyUI 워크플로우 + Civitai LoRA 적용으로 그림체 일관성 향상 검토.

사진 기반 그림일기 — 강아지 사진을 첫 이미지 베이스로 사용 → 자연어 보조 생성. gpt-image-1 vision input 또는 ComfyUI img2img / huggingface FLUX img2img 검토. 진성 심사위원 04-30 ★ 핵심 피드백 정렬.

보안 인젝션 대응 — SQL Injection (SQLAlchemy ORM 대부분 방어, 네이티브 쿼리 점검) + Prompt Injection (사용자 입력 sanitize, 시스템 프롬프트 우회 방어) + OWASP API Top 10 1회 점검. 김효원 심사위원 04-30 전체 팀 공통 권고 정렬.

프롬프트 yaml/config 분리 — 멘토 04-18 액션. ai/prompts/templates/{diary,places}.yaml + ai/prompts/data/{diary\_tone,breeds}.yaml 로 자연어 텍스트 자산을 코드에서 분리해 비개발자도 톤·문구를 편집할 수 있도록 함.

DB 통합 검토 — PostgreSQL + pgvector 단일화 가능성 검토(김효원 심사위원 권고). 통합 진행 또는 '검토했지만 X 이유로 분리 유지' 결정 후 슬라이드 1줄 명시.

본 산출물의 소스코드 권위는 GitHub SKN23-FINAL-3Team 저장소이며, 본문에서 언급된 모듈 경로(ai/, back/api/, ai/evaluation/, ai/intent/) 는 모두 해당 저장소 기준이다. 평가 결과 xlsx 와 RAG 파이프라인 다이어그램 PNG 를 부속 자료로 함께 제출한다.